

Projekt Gruppe 6 - Einführung in Software Engineering

Projekt Name: Hatch-Heroes

Projekt Inhalt (grobe Definition):

Du bekommst ein Ei, dieses musst du ausbrüten, dich um das Tier kümmern und es mit „love points“ oder einem „affection meter“ glücklich halten damit es sich schlussendlich weiterentwickelt. Wenn das Tier genug entwickelt ist, kann man gegen andere „böse“ Tiere kämpfen und Geld verdienen.

Requirements:

Players hatch a creature from an egg and care for it by feeding, washing, playing with it, and letting it sleep. Good care increases its happiness and allows it to evolve. After its **second evolution**, the creature can fight hostile creatures in turn-based battles. There are **no levels or experience points**: development depends on care and happiness.

The game uses a modern graphical interface with an animated creature, care buttons, status bars, and a separate battle screen.

Victoria Hausegger

Functional:

FR1.1

The player shall be able to name an egg, choose one of the four creature types Fire, Water, Earth, or Wind, and hatch it after a displayed countdown. The interface shall show the resulting creature, its type, and its current development stage.

FR1.2

The player shall be able to feed, wash, entertain, and put the creature to sleep. Each action shall improve the corresponding need—fullness, cleanliness, entertainment, or rest—displayed on a scale from 0 to 100. These values shall decrease over time while the game is running.

Non-Functional:

NFR1.1

Each of the four care actions shall be accessible within a maximum of two clicks or taps from the main pet screen.

Erol Bilyalov

Functional:

FR2.1

The system shall store the creature's name, type, needs, development stage, and evolution progress in a local JSON file (data.json) on the player's computer after every completed action. These values shall be restored when the game is reopened.

FR2.2

The system shall calculate happiness from the four needs and automatically evolve the creature when the defined happiness and duration conditions are met. Evolution shall follow the sequence Egg → baby → first evolution → second evolution, without levels or experience points only according to happiness.

Non-Functional:

NFR2.1

With 10,000 stored battle-history entries, loading the current creature and its 20 most recent battles shall take no longer than two seconds on the agreed project test computer.

Jan Magbual

Functional:

FR3.1

The system shall unlock battles after the second evolution. In each round, the player shall choose one of the creature's four attacks while the opponent chooses one of its four attacks at the same time. The system shall display both creatures' health and, for each round, first the opponent's action and then the player's action. The battle shall end when at least one creature reaches zero health, with the result Win, Loss, or Draw.

FR3.2

The system shall calculate damage using the attack and defence values of both chosen attacks and the type bonuses defined in the battle round algorithm; happiness shall not influence damage. It shall select each opponent attack at random from the opponent's four attacks and save each battle's opponent, date and time, and result in a local JSON file (battles.json) on the player's computer.

Non-Functional:

NFR3.1

During 100 test battles on the agreed project test computer, at least 95% of battle actions shall produce their calculated result within 200 milliseconds, excluding animation time.

Algorithm description

Happiness calculation algorithm

The game calculates the creature's happiness from its four current need values.

Let:

- **F** = Fullness, from 0 to 100
- **C** = Cleanliness, from 0 to 100
- **E** = Entertainment, from 0 to 100
- **R** = Rest, from 0 to 100

The happiness value **H** is calculated as:

$$H = \text{round}((F + C + E + R) / 4)$$

The resulting happiness value is therefore between **0 and 100**.

The calculation is repeated whenever one of the four need values changes.

Evolution algorithm

Evolution is based only on happiness and the amount of continuous time for which the required happiness threshold is maintained.

Egg → Baby

1. The player assigns a valid name to the egg and chooses one of the four creature types (Fire, Water, Earth, Wind).
2. A 60-second countdown begins.
3. When the countdown reaches zero, the development stage changes to Baby.

Baby → First Evolution

1. The system continuously evaluates happiness.
2. If happiness is **70 or higher**, the evolution timer progresses.
3. If happiness falls below **70**, the timer is reset to zero.
4. If happiness remains at least 70 for **5 continuous minutes**, the creature evolves to First Evolution.

First Evolution → Second Evolution

1. The system continuously evaluates happiness.
2. If happiness is **80 or higher**, the evolution timer progresses.
3. If happiness falls below **80**, the timer is reset to zero.

4. If happiness remains at least 80 for **10 continuous minutes**, the creature evolves to Second Evolution.

No character level or experience-point calculation is involved in this algorithm.

Creature types

Every creature has exactly one of the four types **Fire, Water, Earth, or Wind**. The following table defines which type is strong against which:

Type	strong against	weak against	neutral to
Fire	Wind	Water	Earth
Water	Fire	Earth	Wind
Earth	Water	Wind	Fire
Wind	Earth	Fire	Water

In short: Fire > Wind > Earth > Water > Fire, where “>” means “strong against”.

- Only “strong against” gives a bonus. “Weak against” has no effect of its own; it only means that the other creature is strong against this one.
 - Two creatures of the same type are neutral to each other.
 - In addition, there are attacks of the type **Normal**. They match no creature type and never receive a bonus.
-

Attacks

Every attack has a name, a type, an attack value, a defence value, and a flag that marks it as a counter. Each creature type and each opponent has exactly **four attacks**.

At the start of a battle, both creatures have **100 health**. The opponent is chosen at random from the list of opponents.

Battle round algorithm

1. The player selects one of the creature's four attacks. At the same time, the opponent selects one of its four attacks (see opponent-action algorithm).
2. If at least one of the two attacks is a **counter**, the round ends without damage.
3. Each side calculates the values of its own attack:
 - $p1 = 115$ if the creature's type is equal to the attack's type, otherwise 100.
 - $p2 = 120$ if $p1 = 115$ **and** the creature is strong against the other creature, otherwise 100.

- $\text{Attack} = \text{round}(\text{attack value} \times p1 \times p2 / 10000)$
- $\text{Defence} = \text{round}(\text{defence value} \times p1 \times p2 / 10000)$

4. Damage is calculated as:

- $\text{Damage to the opponent} = \max(0, \text{player's Attack} - \text{opponent's Defence})$
- $\text{Damage to the player} = \max(0, \text{opponent's Attack} - \text{player's Defence})$

5. Both damage values are subtracted at the same time. A creature's health cannot fall below zero.

6. The system first displays the opponent's attack and the damage it caused, then the player's attack and the damage it caused.

The bonuses are calculated with whole numbers (115 and 120 instead of 1.15 and 1.2), because decimal numbers lead to rounding errors in JavaScript (e.g. $25 \times 1.15 \times 1.2$ results in 34 instead of 35).

Opponent-action algorithm

In every round, the opponent selects one of its four attacks **at random**, each with the same probability. The selection does not depend on the opponent's health.

To keep automated tests reproducible, the round calculation receives the opponent's chosen attack as an input, so a test can specify a fixed opponent attack.

Battle termination algorithm

After every round, the system evaluates both creatures' health:

- If both creatures have **0 health**, the battle ends with the result **Draw**.
- If only the opponent has **0 health**, the battle ends with the result **Win**.
- If only the player's creature has **0 health**, the battle ends with the result **Loss**.
- Otherwise, the next round begins.

After the battle ends, the system stores the opponent, completion date and time, and battle result in a local JSON file (battles.json) on the player's computer.

A victory increases the player's currency balance by **100 coins**. A draw or a loss does not change the currency balance.

The completed battle is then added to the battle history. A battle that is aborted before it ends (for example by reloading the page) is not stored.

Requirements of data handling

DH1 – Creature identification

The system shall store exactly one player-defined name for the currently active creature.

Verification: Create a creature with a defined name, close and reopen the application, and verify that the same name is displayed.

DH2 – Creature needs

The system shall store the current values for **fullness, cleanliness, entertainment, and rest** as integer values in the range **0 to 100 inclusive**.

Verification: Inspect the persisted creature record and confirm that all four values are present and within the permitted range.

DH3 – Development stage

The system shall store the creature's current development stage as one of the following four values:

Egg, Baby, First Evolution, Second Evolution.

Verification: Inspect the persisted value after each evolution.

DH4 – Evolution progress

The system shall store the continuous duration for which the current evolution happiness condition has been satisfied.

Verification: Maintain the required happiness value, restart the application, and verify that the previously accumulated evolution duration is restored.

DH5 – Player currency

The system shall store the player's current in-game currency balance as a non-negative integer.

Verification: Win a battle, restart the application, and compare the restored currency balance with the balance before closing the application.

DH6 – Battle history opponent

For every completed battle, the system shall store the identifier or name of the opponent.

Verification: Complete a battle and inspect the corresponding battle-history record.

DH7 – Battle history timestamp

For every completed battle, the system shall store the date and time at which the battle ended.

Verification: Complete a battle and compare the stored timestamp with the actual completion time.

DH8 – Battle history result

For every completed battle, the system shall store exactly one result with the value **Win**, **Loss**, or **Draw**.

Verification: Complete one winning, one losing, and one drawn battle and verify the persisted values. A draw can also be produced in an automated test with fixed attacks.

DH9 – Persistence after care actions

The system shall persist all creature-state values affected by a completed care action no later than **1 second after the action has completed**.

Verification: Perform each care action individually and inspect the local JSON file (data.json) in a text editor within one second after completion.

DH10 – Persistence after evolution

The system shall persist the new development stage no later than **1 second after an evolution has occurred**.

Verification: Trigger an evolution and inspect the persisted development stage.

DH11 – Restoration after application restart

When the game is reopened, the system shall restore the most recently persisted creature name, type, need values, development stage, evolution progress, and player currency before the main pet screen becomes interactive.

Verification: Modify the stored state, close the application, reopen it, and compare all restored values with the most recently persisted values.

DH12 – Battle-history capacity

The local JSON file (battles.json) on the player's computer shall support at least **10,000 completed battle-history records** without deleting existing records automatically.

Verification: Populate the file with 10,000 battle records and verify that all records remain accessible. This is done by the automated test in tests/tests.html, which uses a separate test file (test-battles.json) so that the player's battles.json is not changed.

DH13 – Creature type

The system shall store the creature's type as one of the following four values: **Fire, Water, Earth, Wind**.

Verification: Create one creature of each type, inspect the persisted creature record, and confirm that the stored type is the chosen one.